Macronutrient Mass Prediction from Food Images

Senén Marqués

June 17, 2026

1 Aim

The goal of this work is to systematically explore ways in which to predict the macronutrient mass (in grams) from food images. Macronutrient mass (grams of protein, fat and carbohydrate) were chosen as the three targets and they were used to calculate calories. Predictions were made from 1) a single image and 2) no reference object for size in the image. These rules served to limit the scope of the project, to allow for the use of the same dataset, and to make a general solution that is suitable for real world applications.

Achieving SOTA or close to SOTA quality presented a significant challenge. In order to accomplish this, the project adopted an experimental and research driven approach, using the literature to justify decisions and hypotheses, and testing those hypotheses through experimentation, thus adhering to the scientific method.

The source code for this project can be found at https://github.com/LuckyToaster/end_of_degree_project

2 Introduction

The biggest challenge when it comes to using computer vision models to predict nutritional information from food images is that it is difficult to predict mass or volume from food images, using a reference object in images may help models better learn this feature [1]. [1] reported low error in predicting volume from food images, although useful in demonstrating how to create a nutrition dataset (one containing reference objects like coins), did not disclose how they obtained such results and simply called it 'a computer algorithm'. Since no dataset containing images with reference objects annotated with nutritional information was found from an early stage, the present work shifted to make a general purpose solution that would hopefully predict low enough errors that would beat human estimations.

The researched on model architectures for this work was limited to an area of research involved in small and efficient architectures. [2] studied how different filter dimensions affected network accuracy and efficiency among other things, they introduced *SqueezeNet*, a CNN with AlexNet level accuracy and 50x fewer parameters. They showcase three strategies to decrease the number of parameters in a network while preserving the most accuracy. The strategis showcased in [2] are: First, replacing 3x3 filters with pointwise convolutions *"since a 1x1 filter has 9X fewer parameters than a 3x3 filter"*. Second, in order to shrink the parameter count further, decrease the number of input channels or features that lead to 3x3 filters, as they highlight that the number of parameters in a layer are (number of input channels) * (number of filters) * (filter dimensions, eg: 3x3). Thirdly, downsampling late in the network as opposed to early, so that the final activation

maps don't loose too much resolution and information is not lost, which according to [2] is done mainly to preserve accuracy.

The parameter shrinking strategies showcased by [2] led them to come up with the *Fire module*, of which SqueezeNet is mostly comprised of. A Fire module is made up of a *squeeze* phase that uses 1x1 filters to shrink the depth of the output tensor depending on the number of 1x1 filters, and an *expand* phase featuring 1x1 and 3x3 filters. So their fire module has three hyperparameters, the number of 1x1 filters in the squeeze, the number of 1x1 filters in the expand and the number of 3x3 filters in the expand phase. They note that in order to align with their second strategy, they keep the number of filters in the squeeze phase to be less than the sum of all the filters in the expand phase.

Later, [3] introduced the *Squeeze and Excitation* (SE) block as a way to perform feature (channel) recalibration *"to selectively emphasise informative features and suppress less useful ones"* [3]. It used the findings of [2] to make an improved module called the *"Squeeze and Excication block"*, which does feature recalibration by some sort of bypass or residual connection. Similar CNN architectures like [4], [5] and [6] were researched to gain a better knowledge of the evolution of Deep Convolutional Neural Networks and a deeper intuition as to what elements constitute an powerful yet efficient architecture.

3 The Data

3.1 Choosing a dataset

The dataset used was the **MM-Food-100K Dataset** [7], which contains ~100K labeled images of restaurant and home cooked food. It was chosen because the images are labelled with nutrionional information needed for this work, for of its large size, and because of its diversity: featuring different dish sizes, cuisines, lighting, camera-quality ... etc.

Other researched datasets include ACEDATA [8], FooDD [9], Pittsburgh Fast Food Image Dataset [10] and Nutrition5K [11]. All of which, provided interesting insights into what makes a good dataset for the task in hand.

3.2 Loading the data

The first step was to download the *MM-Food-100K Dataset*, to do this, a python script was made to download the dataset csv file and images concurrently and idempotently, meaning that upon each execution, the script would attempt to download any missing images, as it took many hours to download the whole 166 GB of images and it was unrealistic to expect the whole script to finish execution uninterrupted in one go. To ensure maximum speed, the data loading script featured concurrency for networking tasks and parallelism for processing and disk access.

After having a local copy of the dataset, a custom pytorch Dataset class was made to allow obtaining the image and targets when iterating over it. That Dataset class would be passed to a DataLoader and then fed to the model.

Later during training, a bottleneck was found, it was caused by the heavy image transform computations from the DataLoader loading batches into the model. The data loading script was modified to offload the image resizing computation to that data loading stage. After that improvement, batches were fed to the model during training at approximately twice the speed.

Finally, upon revisiting the code months later, it was evident that having almost two hundred gigabytes of images locally was not a good idea, so the data loading script was refactored and improved one final time so that images would be downloaded and resized in memory and then saved to disk. This cut the dataset size into a fraction of what it was before and improved the speed of the script somewhat as it made less I/O operations.

3.3 A Quick EDA

After loading the data, it was manually checked for coherence. 15 samples of the dataset were chosen and their calorie and macronutrient annotations were checked so that the calory and macronutrient annotations matched. Many rows were also inspected to see the type of images present and the quality of the other annotations. Then, calorie and macronutrient histograms were plotted to see which targets hold more weight and if the distribution of the data in the dataset is representative of real data, at least at a glance, which is useful when training a model. Below is a sample of the MM-Food-100K dataset, and the corresponding histograms.

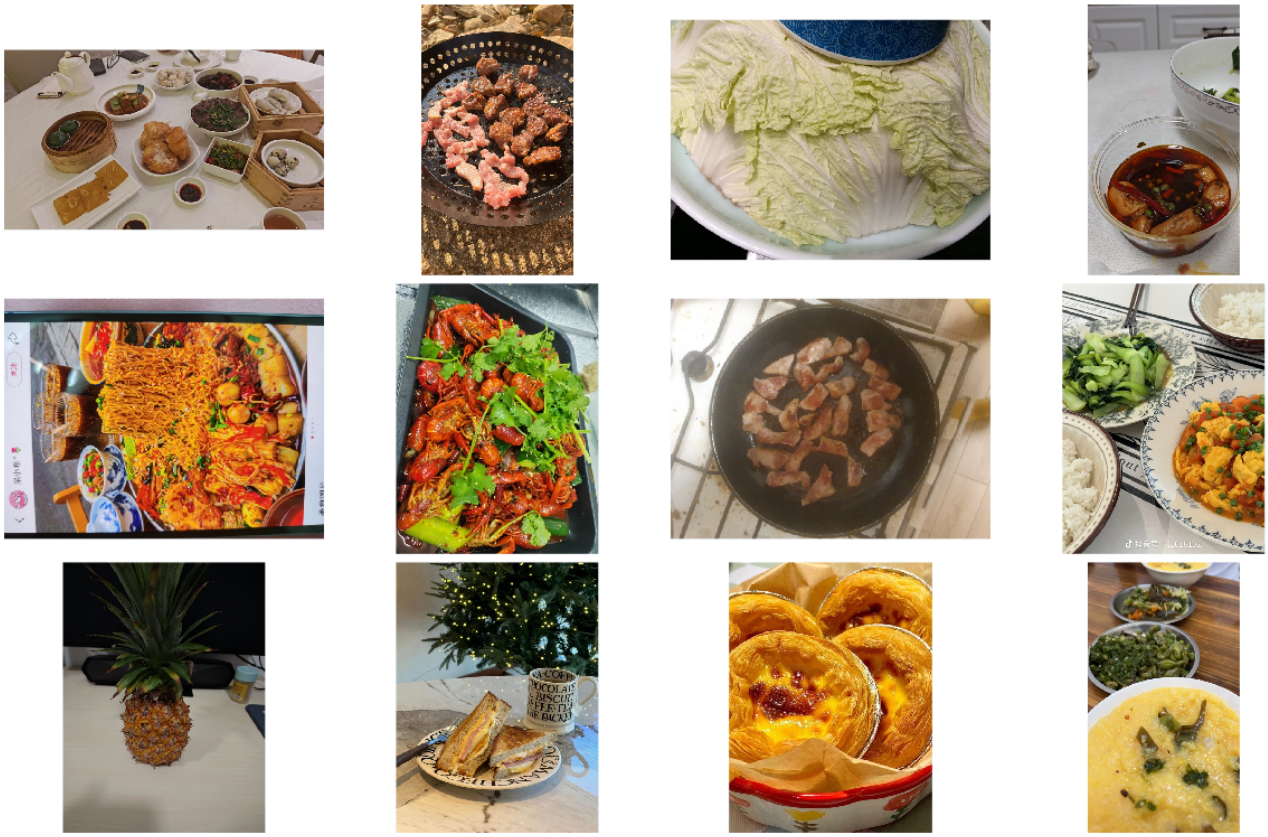

Figure 1: Dataset Sample

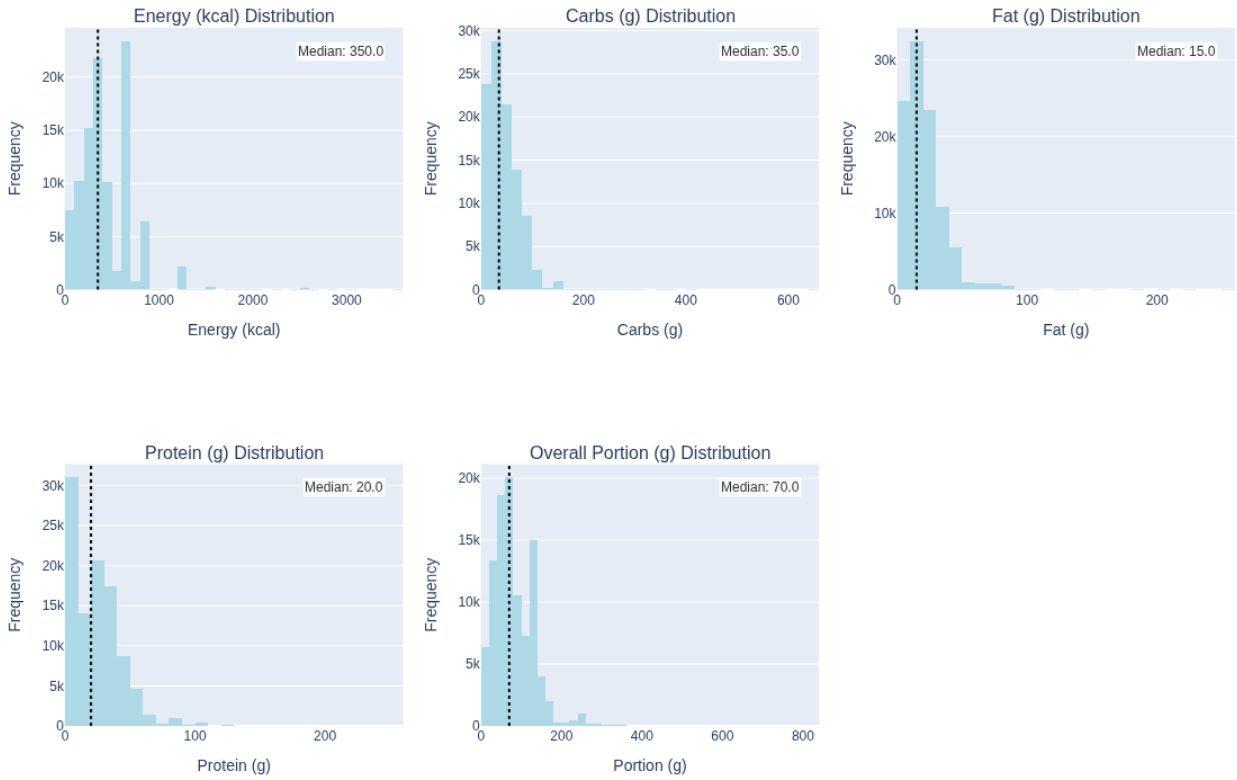

Figure 2: Calorie and Macronutrient Histograms

4 Multi-ouput Regression

The computer vision task at hand requires multi output regression. A Neural Network can be adapted for regression by modifying the head of the network and changing the criterion from Cross-Entropy loss to MSE (L2) or MAE (L1) Loss, *"Standard backbone networks are modified by replacing the final classification layer with a regression head"* [12].

The literature on CNNs is very classification biased, as it is centered around benchmarking on ImageNet classification. After reading much literature on CNN architectures, the question of whether any ImageNet pretrained classification CNN would work well for our regression task emerged. [13] analysed how 16 model architectures perform when fine tuned, trained from scratch or used as feature extractors for 12 different datasets. They found that:

- Imagenet accuracy is a good indicator of transfer accuracy
- On some small, fine grained datasets, the benefit of fine-tuning vs training from scratch is minimal.
- The benefits of ImageNet pretraining fade for larger datasets.
- ImageNet pretraining accelerates convergence.

With this information, it was decided to use transfer learning when training the model, because even if the MMFood100K dataset is too large or too different from ImageNet, it will converge faster cutting down on training time.

4.1 Model Architecture Evaluation

Before using transfer leaning to train a model to do multi-output regression, it was decided to make some empirical experiment to predict which model architectore would perform better.

To find which pretrained model architecture would transfer better, or in other words, which would be the most suitable model for our data and regression task, a *"model shootout"* study was devised.

A study was made with the Optuna library that would pick different pretrained models using grid sampling, train the models with the same hyperparameters, for a fixed number of epochs using feature extraction by freezing the backbone and training only the head.

This was a cheap and fast way to make an educated guess to pick the model since *"when [different models are] used as fixed feature extractors ..., ImageNet top-1 accuracy was correlated with accuracy on transfer learning"* [13] .

However, after seeing the terrible results of said study, and how the models did not learn at all, the *"model shootout"* experiment was updated to run a study for each model to test, for a fixed number of trials, keeping all hyperparameters the same except the most critical, the learning rate. Since how well each architecture learns is mostly affected by the learning rate, it was set as the only hyperparameter to be optimized by optuna, then the best trial for each architecture would be selected and the model architecture with the best learning curve would be picked.

The experiment was ran to test the following models:

- EfficientNet_B3
- EfficientNet_V2_S
- MobileNet_V3_L
- Swin_V2_S

The variant (eg: B3, S, L) of each model was chosen as the biggest variant that could fit on a *16GB VRAM RTX 5060 ti* GPU. After obtaining the results, the Swin vision transformer was chosen because its validation accuracy was slightly higher than the others and because it seemed like the most modern and promising architecture, besides that, there was barely any difference between the learning curves except perhaps a little less overfitting from the Swin model. The results are shown in the figure below.

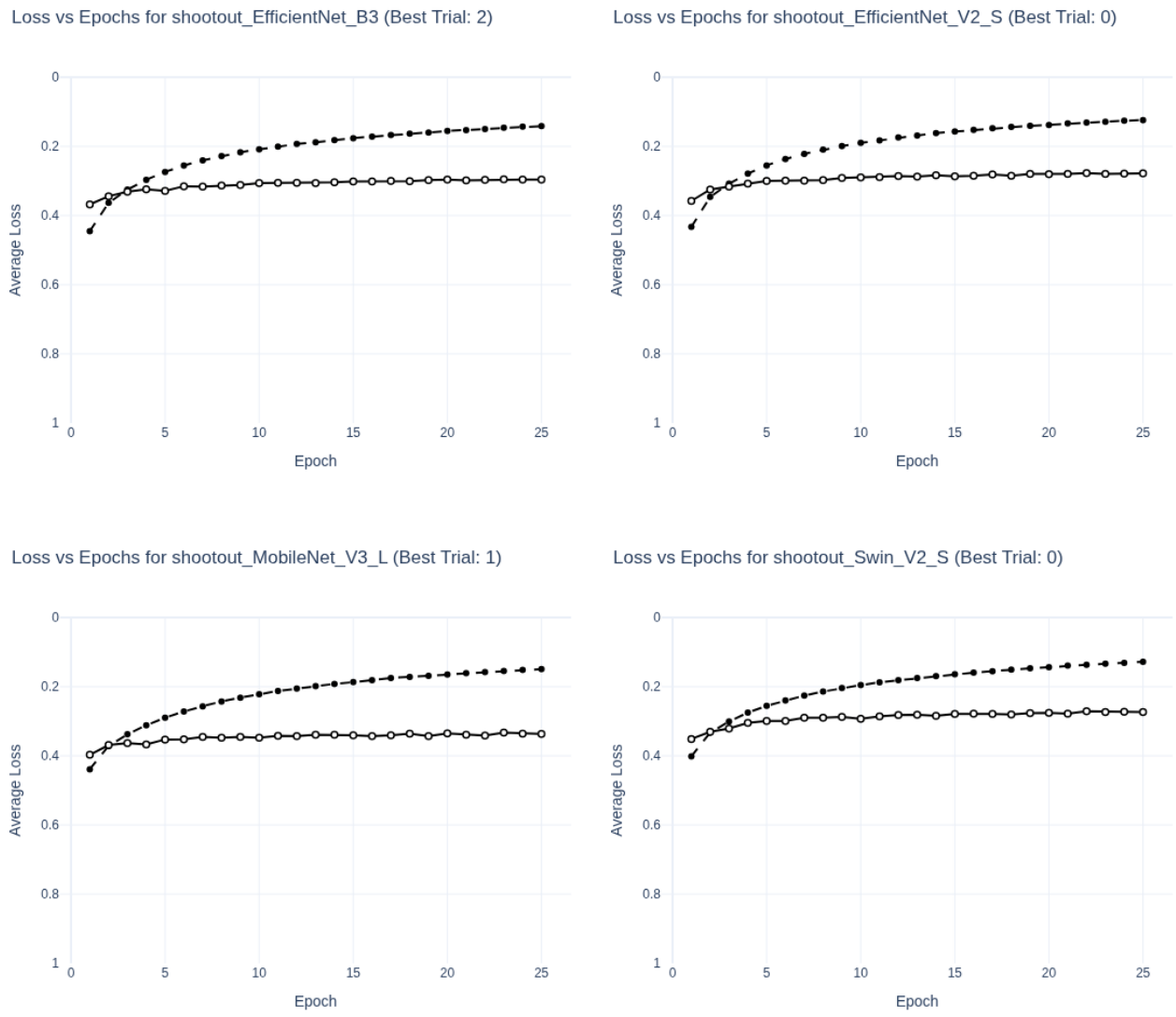

Figure 3: Model shootout results

4.2 Sequential fine tuning with Hyperparameter Optimization

After the Swin model won the shootout, a two step training stage was ran in an extensive hyperparameter optimization study to find the best settings for the model and minimize the loss. The the two step training, or sequential fine tuning, consisted of a *warm up* step before fine tuning, where the head learns while the backbone is frozen. The training loop in both studies featured Automated Mixed Precision Traninig, which was implemented after learning about it through [14] to attempt to train faster and with a lower memory foootprint, and was implemented following an official pytorch tutorial.

The following parameters and hyperparemeters were optimized by the optuna study:

- Feature extraction phase learning rate
- Feature extraction weight decay
- Feature extraction epochs

- Fine tuning phase learning rate
- Fine tuning weight decay
- Fine tuning epochs
- Loss used (MAE, MSE or Huber)
- number of hidden units before the head (not a hyperparameter)
- hidden layer dropout (not a hyperparameter)

On the figure below is the learning curve of the best trial

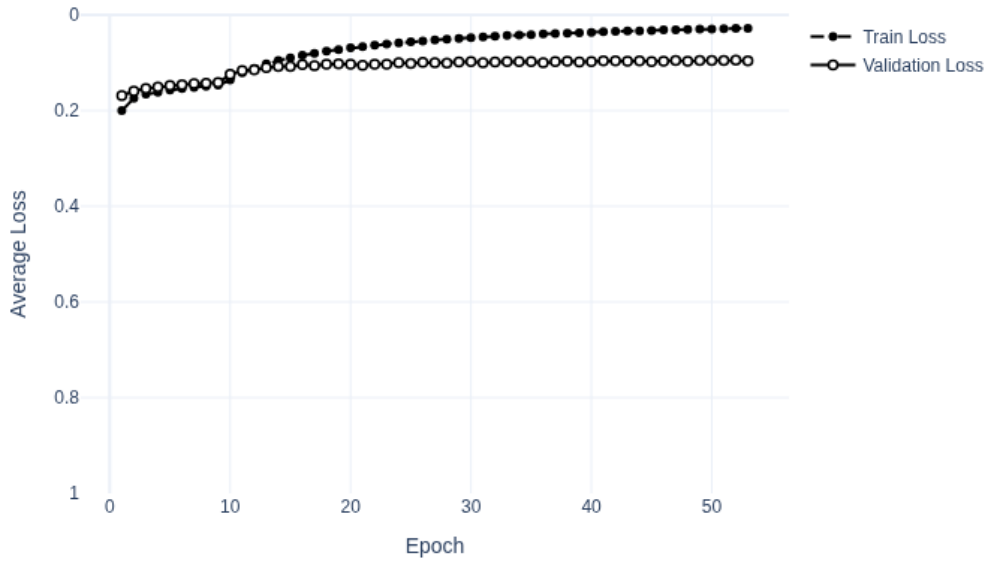

Figure 4: Fine Tuning Best trial

After the study, the model was trained using the best trial parameters and evaluated using a hidden validation split (a 3 way split was carried out throught the project in case the model was overfitted to the validation dataset). The results, are illustrated below.

Nutrient	MAE (grams)	MAE (kcal)
Fat	4.22	37.98
Carbs	9.68	38.72
Protein	4.8	19.2
Total	-	**95.9**

Overall, it was surprising to see s mean average error that low, which suggested that the model could be used as a real world calorie tracking system, albeit a not so precise one, in which a user could over or undereat around 300 calories a day, assuming logging three images daily for breakfast, lunch and dinner. However, these results can beat the nutrion estimates of the average person.

5 LMM enriched with metadata

[8] found that when feeding an image of food enriched with metadata like GPS coordinates, time of day and a list of ingredients to an LMM (Large Multimodal Model) at prompt time, while using prompting techniques like Chain of Thought, Scale Hint in the image, Few-shot and Expert Persona, the error in the predictions would shrink significantly. However, their reported MAE was too high to make such a solution usable.

Nonetheless, the LMM pipeline for estimating nutritional information in [8] was replicated, in order to compare the results with the fine tuned vision transformer. It's inner workings are illustrated in the figure below.

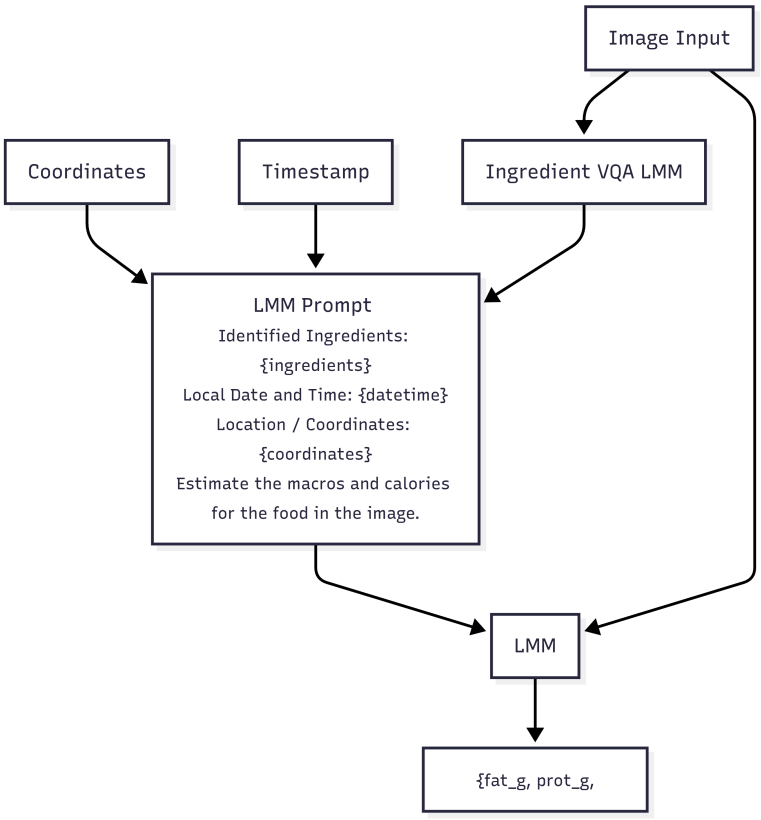

Figure 5: LMM enriched with metadata

It was implemented through the use of the Langchain python module, and it consisted of the following components:

- An ingredient VQA (Visual Question Answering) LMM
- A coordinate fetching function
- A timestamp function
- A nutrition prediction LMM

The ingredient VQA LMM was simply an LMM that is given a food image and outputs a list of detected food items or ingredients. The list of ingredients, location coordinates and timestamp are fed to the nutrition prediction LMM at prompt time to make the final prediction. Google's gemini-3.1-flash-lite LMM was used for both models in the pipeline. Since the food images in the dataset are not annotated with timestamps or information that can be used to infer if the food was meant to be consumed as breakfast, lunch or dinner, the location and time information was not used to test the pipeline. Also, testing the pipeline on the whole hidden validation split would haved incurred thousands of API

calls, which could have turned out too costly, so the evaluation of the pipeline was done with 100 samples of the hidden validation split. The results obtained were practically equivalent to those reported in [8], and can be seen below.

Nutrient	MAE (grams)	MAE (kcal)
Fat	12.47	121.23
Carbs	18.77	75.08
Protein	13.5	54
Total	-	**250.31**

6 Future Improvements

While the results of the fine tuned Swin model were not too unsatisfactory, there is much room for improvement. First, both optuna studies could have been ran for much longer to obtain better results. The results of one promising fine tuning study with over 50 completed trials was accidentally lost, and the second time that the study was ran, the range of the number of epochs was modified mid study, which caused optuna to perform suboptimally, yielding no better studies as it ran sampling the new search space. So it would have been ideal to have both hyperparemeter optimization experiments running properly and for much longer. In addition, better results could be achieved theoretically by removing outliers from the dataset, and performing data augmentations like in [15] and [16]. Moreover, it is highly plausible that the studies for selecting a suitable architecture and fine tuning a model can be further improved. And finally, more experiments could be carried out, perhaps by training more models on different targets, and investigating which combination yields the best results. Some future ideas could be:

- One model to predict each macronutrient mass
- One model to predict total mass and another to predict the distribution of the three macronutrients (although this does not address varying calorie densities of different foods)
- A pipeline like one described in [12] *"[A standard] Pipeline typically consists of food detection, classification, and portion estimation"*

These are important considearations that should be addressed in future work which could yield surprising results.

References

[1] W. Jia *et al.*, "Accuracy of food portion size estimation from digital pictures acquired by a chest-worn camera," *Public Health Nutrition*, vol. 17, no. 8, pp. 1671–1681, 2014, doi: 10.1017/S1368980013003236.

[2] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size." arXiv, 2016. doi: 10.48550/ARXIV.1602.07360.

[3] J. Hu, L. Shen, S. Albanie, G. Sun, and E. Wu, "Squeeze-and-excitation networks." arXiv, 2017. doi: 10.48550/ARXIV.1709.01507.

[4] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications." arXiv, 2017. doi: 10.48550/ARXIV.1704.04861.

[5] A. Howard *et al.*, "Searching for MobileNetV3." arXiv, 2019. doi: 10.48550/ARXIV.1905.02244.

[6] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," 2019, doi: 10.48550/ARXIV.1905.11946.

[7] Y. Dong, Y. Muraoka, S. Shi, and Y. Zhang, "MM-food-100K: A 100,000-sample multimodal food intelligence dataset with verifiable provenance." 2025. Available: https://arxiv.org/abs/2508.10429

[8] B. Coburn, J. He, M. E. Rollo, S. S. Dhaliwal, D. A. Kerr, and F. Zhu, "Comprehensive evaluation of large multimodal models for nutrition analysis: A new benchmark enriched with contextual metadata." 2025. Available: https://arxiv.org/abs/2507.07048

[9] P. Pouladzadeh, A. Yassine, and S. Shirmohammadi, "FooDD: Food detection dataset for calorie measurement using food images," *IEEE Transactions on Instrumentation and Measurement*, vol. 63, no. 8, pp. 1947–1956, 2014, doi: 10.1109/TIM.2014.2323337.

[10] M. Chen, K. Dhingra, W. Wu, L. Yang, R. Sukthankar, and J. Yang, "PFID: Pittsburgh fast-food image dataset," Carnegie Mellon University, CMU-TR-06-09, 2009.

[11] Q. Thames *et al.*, "Nutrition5k: Towards automatic nutritional understanding of generic food." 2021. Available: https://arxiv.org/abs/2103.03375

[12] G. Vinod and F. Zhu, "Food portion estimation: From pixels to calories." 2026. Available: https://arxiv.org/abs/2602.05078

[13] S. Kornblith, J. Shlens, and Q. V. Le, "Do better ImageNet models transfer better?" 2019. Available: https://arxiv.org/abs/1805.08974

[14] P. Micikevicius *et al.*, "Mixed precision training." arXiv, 2017. doi: 10.48550/ARXIV.1710.03740.

[15] E. D. Cubuk, B. Zoph, D. Mane, V. Vasudevan, and Q. V. Le, "AutoAugment: Learning augmentation policies from data." arXiv, 2018. doi: 10.48550/ARXIV.1805.09501.

[16] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, "RandAugment: Practical automated data augmentation with a reduced search space." 2019. Available: https://arxiv.org/abs/1909.13719